Files
main
Go to file
ML3.4 trees
ML3.5 knn
ML3.6 bayes
ML4.1 metrics
ML4.2 threshold
ML4.3 cross-validation
ML4.4 diagnostics
ML4.5 hyperparameters
ML5.1 data integration
data
img
README.md
ML5.2 numeric features
ML5.3 categorical features
ML5.4 target
ML5.5 EDA
ML6.1 clustering
ML6.2 pca
ML7.3 text

koroteev-test Картинки распределены по папкам 7dd1f73 · 2 years ago History

Name	Last commit message	Last commit date
..		
data	Добавлена методичка ML5.1	2 years ago
img	Картинки распределены по папкам	2 years ago
README.md	Картинки распределены по папкам	2 years ago

README.md

Интеграция данных

Цель работы

На практическом примере освоить основные этапы и приемы интеграции разных таблиц данных в единый датасет средствами Python и pandas для последующей подготовки к машинному обучению.

Содержание работы

1. Прочитайте три однотипных файла с данными по обращениям клиентов в службу технической поддержки. Познакомьтесь со структурой этих трех таблиц и их особенностями.
2. Объедините три эти таблицы в один общий датасет с максимальным соблюдением внутренней согласованности данных.
3. Прочитайте датасет с данными заказов клиентов, присоедините максимальное количество данных из него к датасету, получившемуся на предыдущем шаге.
4. Прочитайте датасет с данными о менеджерах службы клиентской поддержки и также присоедините данные из него к общему датасету.

Методические указания

В первую очередь прочитаем первый из трех однотипных табличных файлов, прилагающихся к данной работе:

```
data1 = pd.read_csv('issues1.csv', index_col=0)
data1.head()
```

Мы видим таблицу, состоящую из более чем 28 тысяч строк и 11 колонок:

index	Id	Channel	category	Sub-category	Customer Remarks	Order_id	Issue_reported_Date	Issue_responded
0	e806cdca-d914-43bf-8c9c-ca86c2b38be0	Inbound	Order Related	Order status enquiry	NaN	NaN	03/08/2023 18:23	03/08/2023 20:3
1	a3c94fd3-b620-4b33-9422-25d3719ae3ff	Inbound	Returns	Return request	NaN	NaN	04/08/2023 21:39	04/08/2023 21:4
2	42fc5059-f34b-49b5-b34a-a91a503f0a4e	Inbound	Returns	Reverse Pickup Enquiry	NaN	19dfb969-b591-4262-acab-df6654c0e395	01/08/2023 08:18	01/08/2023 08:2
3	480f7e83-7598-4774-8223-fe767d62021b	Outcall	Order Related	Invoice request	NaN	9d74deb4-ed83-4ed0-840b-e246aa5bfa78	02/08/2023 12:30	02/08/2023 12:3
4	7f62e80e-42c1-4e42-90ac-6c7e6a9fef2e	Inbound	Product Queries	Product Specific Information	Gud ??	d62dbc71-9760-44d8-b865-32bb20edd6fc	01/08/2023 17:52	01/08/2023 18:0

Для работы с данными очень важно понимать предметный смысл каждого значимого атрибута в данных. Здесь мы имеем данные об обращениях клиентов онлайн-магазина в службу технической поддержки. Датасет состоит из 11 колонок:

1. Идентификатор обращения - это уникальная строка, по видимому, генерируемая случайно по алгоритму типа UUID.
2. Канал, по которому обращение поступило в систему. Здесь приводится название канала в текстовом виде.
3. Категория обращения - также дискретная переменная, один вариант из определенного набора.
4. Подкатегория обращения.
5. К обращению пользователи могут прилагать комментарии. В данном поле собраны эти комментарии в простом текстовом, необработанном виде.
6. Идентификатор заказа, по которому сделано обращение. Некоторые значения здесь отсутствуют, то есть обращение может быть связано с заказом, а может и нет.
7. Дата поступления обращения в систему. Здесь фиксируется дата и точное время.

8. Дата ответа на обращение сотрудника техподдержки. Также дата со временем.
9. Дата прохождения клиентом опроса об удовлетворенности работой технической поддержки.
10. Имя сотрудника, который работал с этим обращением.
11. Оценка, которую клиент поставил работе технической поддержки. Именно это и является целевой переменной в данном датасете.

В этой таблице приведены данные свыше 28 тысячах обращений. Но это еще не все, у нас есть еще две подобные таблицы с описанием других обращений. Давайте прочитаем и познакомимся со вторым файлом данных:

```
data2 = pd.read_csv('issues2.csv', index_col=0)
data2
```

В этом файле описаны еще почти 45 тысяч обращений:

index	Id	channel_name	category	Sub-category	Order_id	Reported at	Responded at	Survey responded at	Agent
0	abb64ea1-4512-488c-81f5-51dbb130eec1	Inbound	Returns	Fraudulent User	c9ec6182-a825-4d0f-af28-51be601588e7	08 08 2023, 10:22:00 PM	08 08 2023, 11:42:00 PM	08 08 2023	J. Mcg
1	a1771e0f-9ce1-4ca7-b72d-2ba4e812a901	Inbound	Returns	Return request	55ddd823-2d08-4116-a863-1e6d77a27b23	17 08 2023, 08:15:00 PM	17 08 2023, 09:05:00 PM	17 08 2023	M. Mc
2	e556d65f-885c-42db-b8d2-8d726301910e	Inbound	Order Related	Order status enquiry	76b5dac1-946e-4ef4-aefa-ca9ef12abeed	08 08 2023, 07:54:00 PM	08 08 2023, 07:56:00 PM	08 08 2023	H. Glc
3	f920ef14-be08-4c76-b446-7acacf6321d7	Outcall	Returns	Return request	591ca014-2903-4ac2-a075-8992ecc2abb8	17 08 2023, 02:39:00 PM	17 08 2023, 03:46:00 PM	17 08 2023	S. Nel
4	c6d3d6a3-3ff3-451d-898c-4bdea1c54932	Inbound	Returns	Return request	5967afff-aa5b-445b-8b9b-d625b93a886c	25 08 2023, 11:56:00 AM	25 08 2023, 12:10:00 PM	25 08 2023	N. Ng

Из явно бросающихся в глаза отличий, мы видим отсутствие во втором файле информации о тексте комментария. Очень часто бывает, что в разных датасетах описаны разные характеристики объектов. Может быть, что второй файл поступил их другой системы, где текстовые комментарии просто не собираются или не учитываются. Либо наоборот, обращения без комментариев попали в системе распределения данных в другую таблицу. Спойлер - у нас именно второй случай.

Еще заметим, что некоторые колонки во втором файле называются немного по-другому. Это нам еще будет важно дальше.

Дальше точно также познакомимся с третьей таблицей данных:

```
data3 = pd.read_csv('issues4.csv', index_col=0)
data3.head()
```

Здесь содержится меньше всего - 13 тысяч обращений. И меньше всего колонок - мы не видим ни текста обращения, ни связанного идентификатора заказа:

index	Unique id	channel_name	category	Sub-category	Issue_reported at	issue_responded	Survey_response_Da
0	9273f542-b0e1-411c-bebe-69d25a1cfb81	Inbound	Payments related	Online Payment Issues	August 29, 2023, 05:07:00 PM	August 29, 2023, 06:10:00 PM	August 29, 2023, 12:00:00 AM
1	247b304e-fcf5-4e44-bc72-15437056299a	Inbound	Order Related	Order status enquiry	August 16, 2023, 03:34:00 PM	August 16, 2023, 03:42:00 PM	August 16, 2023, 12:00:00 AM
2	d8ac7d94-b1ae-4316-81ef-4199d936f971	Outcall	Order Related	Installation/demo	August 14, 2023, 12:22:00 PM	August 14, 2023, 01:10:00 PM	August 14, 2023, 12:00:00 AM
3	40e0781c-c81d-4646-9f1c-0a3cbe569596	Outcall	Returns	Return request	January 08, 2023, 08:24:00 PM	January 08, 2023, 08:31:00 PM	August 01, 2023, 12:00:00 AM
4	5bb8cfb5-9813-4710-adef-1042a0a16d0b	Inbound	Returns	Missing	August 16, 2023, 08:05:00 AM	August 16, 2023, 12:23:00 PM	August 16, 2023, 12:00:00 AM

Еще можно обратить внимание на другой формат дат (хотя во всех трех файлах он разный), и на полное указание имени оператора техподдержки. В двух предыдущих файлах имя оператора указывалось инициалом.

Выведем формы всех трех получившихся таблиц:

```
data1.shape, data2.shape, data3.shape
```

```
((28425, 11), (44339, 10), (13143, 9))
```

Теперь приступим к объединению таблиц в единый набор данных. В первую очередь нам нужно определить тип интеграции. Так как во всех трех наборах представлена примерно одинаковая информация о разных объектах предметной области, нам нужно вертикальное объединение таблиц.

Вертикальная интеграция данных

Важно любую интеграцию данных проводить последовательно, не стараясь ускорить процесс массовым объединением многих таблиц. Начнем с объединения первой и второй таблицы. Вертикальное объединение осуществляется простой конкатенацией датасетов. Но если мы проведем конкатенацию сразу, вот так:

```
pd.concat([data1, data2]).head()
```

мы получим не тот результат, который нужно:

index	Id	Channel	category	Sub-category	Customer Remarks	Order_id	Issue_reported_Date	Issue_responde
0	03f3aaf6-85c5-4eb9-af30-7ab7cc7b627d	Inbound	Cancellation	Not Needed	Food	b0a0dece-5fda-4c0f-a69c-e6bcde3de8d4	2023-08-29 08:02:00	2023-08-29 08:
1	f0442a16-b6d7-4fce-87fa-cbff2591176c	Outcall	Returns	Return request	V nicely	NaN	2023-03-08 18:38:00	2023-03-08 19:
2	99b7f6fa-a7b1-41a6-86ad-711ca8658fca	Inbound	Returns	Reverse Pickup Enquiry	Good	96e2ed8b-88b8-4768-a7e4-9468275c8085	2023-08-28 11:00:00	2023-08-28 00:
3	95dd50e0-bdbd-4e99-9278-e139a76a94d3	Inbound	Order Related	Delayed	Thank you for help	4340c8ed-d80f-4712-921a-4fea0d60dcc6	2023-05-08 23:42:00	2023-06-08 02:
4	2af35e04-d13b-43e9-9e69-358568f6174a	Inbound	Offers & Cashback	Affiliate Offers	Good	1549b443-c2c1-47b2-8608-cd4d5576d288	2023-01-08 11:13:00	2023-01-08 11:

Здесь мы видим, что из-за разности наименования столбцов у нас получилось дублирование: столбцы из второй таблицы не стали продолжением столбцов из первой, а просто добавились в качестве новых. При этом по строкам из первой таблицы в этих столбцах автоматически проставлены пропуски. Такую ситуацию можно сразу отследить по количеству столбцов в итоговом датафрейме. Их должно было быть 11, а стало 1 - явно что-то пошло не так.

Выведем названия столбцов в первой таблице:

```
data1.info()
```

```
Int64Index: 28425 entries, 0 to 28424
Data columns (total 11 columns):
 #   Column               Non-Null Count  Dtype 
---  ------               --------------  ----- 
 0   Id                   28425 non-null  object
 1   Channel              28425 non-null  object
 2   category             28425 non-null  object
 3   Sub-category         28425 non-null  object
 4   Customer Remarks     28425 non-null  object
 5   Order_id             22387 non-null  object
 6   Issue_reported_Date  28425 non-null  object
 7   Issue_responded_Date  28425 non-null  object
 8   Survey_response_Date  28425 non-null  object
 9   Agent_name           28425 non-null  object
 10  CSAT Score           28425 non-null  int64 
dtypes: int64(1), object(10)
memory usage: 3.6+ MB
```

И во второй:

```
data2.info()
```

```
Int64Index: 44339 entries, 0 to 44338
Data columns (total 10 columns):
 #   Column              Non-Null Count  Dtype 
---  ------              --------------  ----- 
 0   Id                  44339 non-null  object
 1   channel_name        44339 non-null  object
 2   category            44339 non-null  object
 3   Sub-category        44339 non-null  object
 4   Order_id            44271 non-null  object
 5   Reported at         44339 non-null  object
 6   Responded at        44339 non-null  object
```

```
 7   Survey responded at  44339 non-null  object
 8   Agent_name           44339 non-null  object
 9   CSAT Score           44339 non-null  int64
dtypes: int64(1), object(9)
memory usage: 4.7+ MB
```

Отсюда можно построить схему переименования колонок так, чтобы в обеих таблицах одинаковые по смыслу колонки назывались в точности идентично, например так:

```
data2 = data2.rename(columns={
    'channel_name': 'Channel',
    'Reported at': 'Issue_reported_Date',
    'Responded at': 'Issue_responded_Date',
    'Survey responded at': 'Survey_response_Date',
})
```

Но на этом проблемы интеграции не заканчиваются. Еще важно следить за тем, чтобы в обеих таблицах были абсолютно одинаковые форматы. Лучший способ этого добиться - перевести все даты во внутренний формат pandas - datetime. Для этого текстовое представление дат нужно распарсить. Обычно, метод to_datetime хорошо справляется с этим автоматически, но иногда приходится указывать конкретный формат даты:

```
data2['Issue_reported_Date'] = pd.to_datetime(data2['Issue_reported_Date'])
data2['Issue_responded_Date'] = pd.to_datetime(data2['Issue_responded_Date'])
data2['Survey_response_Date'] = pd.to_datetime(data2['Survey_response_Date'], format='%d %m %Y')
```

После выполнения этих манипуляций сигнатура второй таблицы (набор названий колонок) полностью соответствует первой:

```
data2.info()
```

Обратите внимание на тип колонок с датами:

```
Int64Index: 44339 entries, 0 to 44338
Data columns (total 10 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   Id                    44339 non-null  object
 1   Channel               44339 non-null  object
 2   category              44339 non-null  object
 3   Sub-category          44339 non-null  object
 4   Order_id              44271 non-null  object
 5   Issue_reported_Date   44339 non-null  datetime64[ns]
 6   Issue_responded_Date  44339 non-null  datetime64[ns]
 7   Survey_response_Date  44339 non-null  datetime64[ns]
 8   Agent_name            44339 non-null  object
 9   CSAT Score            44339 non-null  int64
dtypes: datetime64[ns](3), int64(1), object(6)
memory usage: 4.7+ MB
```

И само содержание таблицы тоже приведено в соответствие с соглашениями, которые приняты в первой:

```
data2.head()
```

index	Id	Channel	category	Sub-category	Order_id	Issue_reported_Date	Issue_responded_Date	Surv
0	abb64ea1-4512-488c-81f5-51dbb130eec1	Inbound	Returns	Fraudulent User	c9ec6182-a825-4d0f-af28-51be601588e7	2023-08-08 22:22:00	2023-08-08 23:42:00	202
1	a1771e0f-9ce1-4ca7-b72d-2ba4e812a901	Inbound	Returns	Return request	55ddd823-2d08-4116-a863-1e6d77a27b23	2023-08-17 20:15:00	2023-08-17 21:05:00	202
2	e556d65f-885c-42db-b8d2-8d726301910e	Inbound	Order Related	Order status enquiry	76b5dac1-946e-4ef4-aefa-ca9ef12abeed	2023-08-08 19:54:00	2023-08-08 19:56:00	202
3	f920ef14-be08-4c76-b446-7acacf6321d7	Outcall	Returns	Return request	591ca014-2903-4ac2-a075-8992ecc2abb8	2023-08-17 14:39:00	2023-08-17 15:46:00	202
4	c6d3d6a3-3ff3-451d-898c-4bdea1c54932	Inbound	Returns	Return request	5967afff-aa5b-445b-8b9b-d625b93a886c	2023-08-25 11:56:00	2023-08-25 12:10:00	202

В первой таблице также надо преобразовать даты в нужный формат:

```
data1['Issue_reported_Date'] = pd.to_datetime(data1['Issue_reported_Date'])
data1['Issue_responded_Date'] = pd.to_datetime(data1['Issue_responded_Date'])
data1['Survey_response_Date'] = pd.to_datetime(data1['Survey_response_Date'], format='%d-%b-%y')
```

Конечно, мы пропускаем множество шагов, необходимых для полного исключения возможных ошибок в интеграции. Например, нужно убедиться, что по всем остальным колонкам соблюдаются общие соглашения и обозначения. Например, в категориальных атрибутах таких как название канала, мы не уверены в соответствии названий, которые есть в первой таблице, названиям во второй.

По хорошему, надо проверять все значения категориальных колонок по смыслу. И конечно, это невозможно правильно сделать без глубокого понимания предметной области.

Чаще всего, интеграция разных таблиц требует именно взаимной подгонки, то есть преобразования и в первой и во второй части. Убедимся, что сигнатура первой таблицы соответствует:

```
data1.info()
```

```
Int64Index: 28425 entries, 0 to 28424
Data columns (total 11 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   Id                   28425 non-null  object
 1   Channel              28425 non-null  object
 2   category             28425 non-null  object
 3   Sub-category         28425 non-null  object
 4   Customer Remarks     28425 non-null  object
 5   Order_id             22387 non-null  object
 6   Issue_reported_Date  28425 non-null  datetime64[ns]
 7   Issue_responded_Date 28425 non-null  datetime64[ns]
 8   Survey_response_Date 28425 non-null  datetime64[ns]
 9   Agent_name           28425 non-null  object
 10  CSAT Score           28425 non-null  int64
dtypes: datetime64[ns](3), int64(1), object(7)
memory usage: 3.6+ MB
```

Вот теперь можно провести собственно объединение данных:

```
data_12 = pd.concat([data1, data2])
data_12.head()
```

Обязательно убедимся в правильность полученной таблицы:

index	Id	Channel	category	Sub-category	Customer Remarks	Order_id	Issue_reported_Date	Issue_responde
0	03f3aaf6-85c5-4eb9-af30-7ab7cc7b627d	Inbound	Cancellation	Not Needed	Food	b0a0dece-5fda-4c0f-a69c-e6bcde3de8d4	2023-08-29 08:02:00	2023-08-29 08:
1	f0442a16-b6d7-4fce-87fa-cbff2591176c	Outcall	Returns	Return request	V nicely	NaN	2023-03-08 18:38:00	2023-03-08 19:
2	99b7f6fa-a7b1-41a6-86ad-711ca8658fca	Inbound	Returns	Reverse Pickup Enquiry	Good	96e2ed8b-88b8-4768-a7e4-9468275c8085	2023-08-28 11:00:00	2023-08-28 00:
3	95dd50e0-bdbd-4e99-9278-e139a76a94d3	Inbound	Order Related	Delayed	Thank you for help	4340c8ed-d80f-4712-921a-4fea0d60dcc6	2023-05-08 23:42:00	2023-06-08 02:
4	2af35e04-d13b-43e9-9e69-358568f6174a	Inbound	Offers & Cashback	Affiliate Offers	Good	1549b443-c2c1-47b2-8608-cd4d5576d288	2023-01-08 11:13:00	2023-01-08 11:

Отдельно проверим, что не появилось никаких лишних колонок:

```
data_12.info()
```

```
Int64Index: 72764 entries, 0 to 44338
Data columns (total 11 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   Id                   72764 non-null  object
 1   Channel              72764 non-null  object
 2   category             72764 non-null  object
 3   Sub-category         72764 non-null  object
 4   Customer Remarks     28425 non-null  object
 5   Order_id             66658 non-null  object
 6   Issue_reported_Date  72764 non-null  datetime64[ns]
 7   Issue_responded_Date 72764 non-null  datetime64[ns]
 8   Survey_response_Date 72764 non-null  datetime64[ns]
 9   Agent_name           72764 non-null  object
 10  CSAT Score           72764 non-null  int64
dtypes: datetime64[ns](3), int64(1), object(7)
memory usage: 6.7+ MB
```

После этого можно попробовать присоединить и третью часть. Также выведем основную информацию по колонкам третьей таблицы:

```
data3.info()
```

```
Int64Index: 13143 entries, 0 to 13142
Data columns (total 9 columns):
 #   Column               Non-Null Count  Dtype
```

```
---  ------              --------------  -----
 0   Unique id           13143 non-null  object
 1   channel_name        13143 non-null  object
 2   category            13143 non-null  object
 3   Sub-category        13143 non-null  object
 4   Issue_reported at   13143 non-null  object
 5   issue_responded     13143 non-null  object
 6   Survey_response_Date  13143 non-null  object
 7   Agent               13143 non-null  object
 8   CSAT Score          13143 non-null  int64
dtypes: int64(1), object(8)
memory usage: 1.5+ MB
```

Также, как и на предыдущем шаге, колонки требуют переименования:

```
data3 = data3.rename(columns={
    'Unique id': 'Id',
    'channel_name': 'Channel',
    'Issue_reported at': 'Issue_reported_Date',
    'issue_responded': 'Issue_responded_Date',
    'Survey responded at': 'Survey_response_Date',
    'Agent': 'Agent_name'
})
```

И также парсим все даты:

```
data3['Issue_reported_Date'] = pd.to_datetime(data3['Issue_reported_Date'])
data3['Issue_responded_Date'] = pd.to_datetime(data3['Issue_responded_Date'])
data3['Survey_response_Date'] = pd.to_datetime(data3['Survey_response_Date'])
```

Убеждаемся, что третья таблица правильно преобразовалась:

```
data3.head()
```

index	Id	Channel	category	Sub-category	Issue_reported_Date	Issue_responded_Date	Survey_respon
0	9273f542-b0e1-411c-bebe-69d25a1cfb81	Inbound	Payments related	Online Payment Issues	2023-08-29 17:07:00	2023-08-29 18:10:00	2023-08-29 00:
1	247b304e-fcf5-4e44-bc72-15437056299a	Inbound	Order Related	Order status enquiry	2023-08-16 15:34:00	2023-08-16 15:42:00	2023-08-16 00:
2	d8ac7d94-b1ae-4316-81ef-4199d936f971	Outcall	Order Related	Installation/demo	2023-08-14 12:22:00	2023-08-14 13:10:00	2023-08-14 00:
3	40e0781c-c81d-4646-9f1c-0a3cbe569596	Outcall	Returns	Return request	2023-01-08 20:24:00	2023-01-08 20:31:00	2023-08-01 00:
4	5bb8cfb5-9813-4710-adef-1042a0a16d0b	Inbound	Returns	Missing	2023-08-16 08:05:00	2023-08-16 12:23:00	2023-08-16 00:

Выводим измененную сигнатуру третьей таблицы:

```
data3.info()
```

```
Int64Index: 13143 entries, 0 to 13142
Data columns (total 9 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   Id                    13143 non-null  object
 1   Channel               13143 non-null  object
 2   category              13143 non-null  object
 3   Sub-category          13143 non-null  object
 4   Issue_reported_Date   13143 non-null  datetime64[ns]
 5   Issue_responded_Date  13143 non-null  datetime64[ns]
 6   Survey_response_Date  13143 non-null  datetime64[ns]
 7   Agent_name            13143 non-null  object
 8   CSAT Score            13143 non-null  int64
dtypes: datetime64[ns](3), int64(1), object(5)
memory usage: 1.5+ MB
```

Можно проводить объединение:

```
data_123 = pd.concat([data_12, data3])
data_123.head()
```

Убеждаемся, что ничего неожиданного не произошло:

```
data_123.info()
```

```
Int64Index: 85907 entries, 0 to 13142
Data columns (total 11 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   Id                    85907 non-null  object
 1   Channel               85907 non-null  object
 2   category              85907 non-null  object
 3   Sub-category          85907 non-null  object
 4   Customer Remarks      28425 non-null  object
 5   Order_id              66658 non-null  object
 6   Issue_reported_Date   85907 non-null  datetime64[ns]
 7   Issue_responded_Date  85907 non-null  datetime64[ns]
 8   Survey_response_Date  85907 non-null  datetime64[ns]
 9   Agent_name            85907 non-null  object
 10  CSAT Score            85907 non-null  int64
dtypes: datetime64[ns](3), int64(1), object(7)
memory usage: 7.9+ MB
```

Выводим первые строки таблицы:

```
data_123.head()
```

Здесь можно заметить одну ошибку, которую мы пропустили. Обратите внимание на значения в колонке "Agent_name":

index	Id	Channel	category	Sub-category	Customer Remarks	Order_id	Issue_reported_Date	Issue_responde
0	03f3aaf6-85c5-4eb9-af30-7ab7cc7b627d	Inbound	Cancellation	Not Needed	Food	b0a0dece-5fda-4c0f-a69c-e6bcde3de8d4	2023-08-29 08:02:00	2023-08-29 08:
1	f0442a16-b6d7-4fce-87fa-cbff2591176c	Outcall	Returns	Return request	V nicely	NaN	2023-03-08 18:38:00	2023-03-08 19:
2	99b7f6fa-a7b1-41a6-86ad-711ca8658fca	Inbound	Returns	Reverse Pickup Enquiry	Good	96e2ed8b-88b8-4768-a7e4-9468275c8085	2023-08-28 11:00:00	2023-08-28 00:
3	95dd50e0-bdbd-4e99-9278-e139a76a94d3	Inbound	Order Related	Delayed	Thank you for help	4340c8ed-d80f-4712-921a-4fea0d60dcc6	2023-05-08 23:42:00	2023-06-08 02:
4	2af35e04-d13b-43e9-9e69-358568f6174a	Inbound	Offers & Cashback	Affiliate Offers	Good	1549b443-c2c1-47b2-8608-cd4d5576d288	2023-01-08 11:13:00	2023-01-08 11:

В первых двух таблицах у нас имя оператора обозначено инициалом, а в третьей - полным именем. Необходимо привести все данные к единому соглашению. Здесь нужно приводить к форме с инициалом, потому что полное имя мы можем сократить до инициала, а только по инициалу восстановить полное имя нельзя. Напишем процедуру, которая преобразует строку нужным нам образом и применим ее ко всей колонке в третьей таблице.

```
data3['Agent_name'] = data3['Agent_name'].apply(
    lambda x: x.split()[0][0] + ". " + x.split()[1] if isinstance(x, str) else np.NAN
)
```

Такие операции могут выполняться довольно долго, так как они не оптимизированы. Но по завершении обработки нам останется только повторить присоединение третьей таблицы:

```
data_123 = pd.concat([data_12, data3])
```

Таким образом мы правильно провели вертикальное объединение трех таблиц, содержащих информацию о разных объектах в единый датасет. В реальном примере нужно еще более тщательно подходить к проверке внутренней согласованности значений в таблицах, желательно при этом выводить статистическую информацию по столбцам в каждой таблице, можно строить графики плотности распределения, гистограммы. А после объединения - обязательно проверить, не получилось ли в итоговой таблице дубликатов - нескольких записей по одному и тому же объекту.

Добавление данных заказа

Итак, у нас собраны в одной таблице данные по всем имеющимся обращениям клиентов в техподдержку. Мы могли заметить, что среди полей в этой таблице есть идентификатор заказа. Информация по заказам хранится в отдельной таблице. Прочитаем ее и выведем на экран первые строки:

```
orders_data = pd.read_csv('orders.csv', index_col=0)
orders_data.head()
```

index	Id	order_date_time	Customer_City	Product_category	Item_price
0	5779ad15-5147-438c-a7eb-94b44a79b610	14/07/2023 16:07	AMBEGAON	LifeStyle	145.0
1	5db8efa3-4cdc-4eff-9b97-7ff8ca20dcbd	09/08/2023 21:15	BULANDSHAHR	Electronics	1599.0
2	5f678412-588d-4a8a-96a8-ef5998c5f34f	05/07/2023 12:53	SAHARSA	Electronics	41990.0
3	786444c1-d3a7-473e-94a3-461fa635339b	15/07/2023 13:49	BULDHANA	Electronics	2699.0
4	58e5dae7-2b28-4f1e-9347-384a172b3469	06/08/2023 15:52	RAJKOT	LifeStyle	183.0

Первый же столбец - это идентификатор заказа, который должен совпадать с идентификатором с общей таблице. Для проведение соединения таблиц в pandas необязательно, но очень облегчает процедуру, если соответствующие столбцы таблиц назывались одинаково. Поэтому переименуем столбец в данной таблице так, чтобы он назывался как в основной:

```
orders_data = orders_data.rename(columns={
    'Id': 'Order_id',
})
```

Теперь можно провести соединение. Для этого выделим ключевой столбец - тот, по которому будет проводиться сопоставление записей. Самое важное - выбрать тип соединения. В данном случае, нам нужно выбрать все записи из первой (левой) таблицы и присоединить к ним соответствующие столбцы из второй (правой) таблицы. Значит, нужно использовать левое внешнее соединение:

```
data_with_orders = data_123.merge(orders_data, on='Order_id', how='left')
data_with_orders.head()
```

При внимательном рассмотрении данной таблицы можно заметить, что для многих строк в новых столбцах стоят пропуски, то отсутствие значения:

index	Id	Channel	category	Sub-category	Customer Remarks	Order_id	Issue_reported_Date	Issue_responde
0	03f3aaf6-85c5-4eb9-af30-7ab7cc7b627d	Inbound	Cancellation	Not Needed	Food	b0a0dece-5fda-4c0f-a69c-e6bcde3de8d4	2023-08-29 08:02:00	2023-08-29 08:
1	f0442a16-b6d7-4fce-87fa-cbff2591176c	Outcall	Returns	Return request	V nicely	NaN	2023-03-08 18:38:00	2023-03-08 19:
2	99b7f6fa-a7b1-41a6-86ad-711ca8658fca	Inbound	Returns	Reverse Pickup Enquiry	Good	96e2ed8b-88b8-4768-a7e4-9468275c8085	2023-08-28 11:00:00	2023-08-28 00:
3	95dd50e0-bdbd-4e99-9278-e139a76a94d3	Inbound	Order Related	Delayed	Thank you for help	4340c8ed-d80f-4712-921a-4fea0d60dcc6	2023-05-08 23:42:00	2023-06-08 02:
4	2af35e04-d13b-43e9-9e69-358568f6174a	Inbound	Offers & Cashback	Affiliate Offers	Good	1549b443-c2c1-47b2-8608-cd4d5576d288	2023-01-08 11:13:00	2023-01-08 11:

Можно подумать, что соединение прошло не так, как мы предполагали. Можно вручную проверить некоторые значения. Для этого подставим идентификатор заказа, по которому нет данных в итоговой таблице в таблицу заказов:

```
orders_data[orders_data.Order_id == 'a5464619-b2d8-49de-9346-5e5db0972756']
```

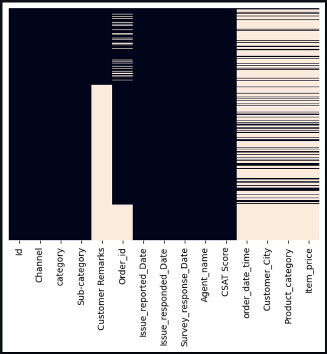
Мы видим, что действительно по данному идентификатору в таблице заказов нет ни одной записи:

index	Order_id	order_date_time	Customer_City	Product_category	Item_price

Значит, соединение прошло правильно, просто в таблице аказов есть информация не про каждый заказ, упомянутый в основной таблице. Оценить масштаб пропусков в данных можно с помощью визуализации отсутствующих значений:

```
sns.heatmap(data_with_orders.isnull(), yticklabels=False, cbar=False)
```

Да данном графике видны, в частности, пропущенные значения в столбцах с комментарием пользователя и идентификатором заказа, которые появились после вертикальной интеграции первых трех таблиц. А по данным заказа мы видим большое количество пропусков. Но там, где нет идентификатора заказа, нет и никакой информации про сам заказ, что логично.



На последующих шагах анализа и очистки данных потребуется избавится от этих пропусков. Вполне возможно,что целесообразнее изначально не присоединять эту таблицу, так как она содержит информацию только о малой части заказов. Но эти решения мы будем обсуждать позже. Сейчас наша задача в том, чтобы объединить в одну таблицу максимальное количество данных.

Добавление данных менеджеров

Перейдем к последней таблице нашего набора - данным об операторах технической поддержки. Как всегда первым делом прочтем и выведем таблицу:

```
agent_data = pd.read_csv("agents.csv", index_col=0)
agent_data.head()
```

index	Agent	Supervisor	Manager	Tenure Bucket	Shift
0	Aaron Edwards	Mia Patel	Emily Chen	61-90	Evening
1	Aaron Romero	Mason Gupta	Jennifer Nguyen	On Job Training	Morning
2	Abigail Gonzalez	Jacob Sato	Jennifer Nguyen	On Job Training	Morning
3	Adam Barnett	Abigail Suzuki	Jennifer Nguyen	On Job Training	Morning
4	Adam Hammond	Olivia Suzuki	John Smith	31-60	Morning

В этой таблице собрана информация о более чем 1300 работниках технической поддержки, которые обрабатывают обращения клиентов. У каждого оператора есть руководитель и менеджер, а также тип контракта и вид смены. В нашей основной таблице есть поле с именем оператора. Похоже, что именно по нему и надо будет проводить соединение. Но для начала нужно убедиться, что в этой новой таблице имя оператора уникальное. Для этого проведем группировку по имени оператора и выведем количество значений по остальным столбцам:

```
agent_data.groupby(['Agent']).agg('count').head(20)
```

Видно, что каждому имени оператора соответствует только одно значение по всем другим столбцам:

Agent	Supervisor	Manager	Tenure Bucket	Shift
Aaron Edwards	1	1	1	1
Aaron Romero	1	1	1	1
Abigail Gonzalez	1	1	1	1
Adam Barnett	1	1	1	1
Adam Hammond	1	1	1	1
Adam Henderson	1	1	1	1
Adam Hernandez	1	1	1	1
Adam Schwartz	1	1	1	1
Adam Torres	1	1	1	1
Adrian Branch	1	1	1	1
Adrian Johnson	1	1	1	1
Adrian Richards	1	1	1	1
Adriana Adams	1	1	1	1
Alan Cruz	1	1	1	1
Alan Davies	1	1	1	1
Alan Sweeney	1	1	1	1
Albert Thornton	1	1	1	1
Alexa Mcpherson	1	1	1	1

Alexander Marshall	1	1	1	1
Alexander Mata	1	1	1	1

Однако, в нашей основной таблице операторы названы через инициалы. Давайте в этой таблице также приведем имя оператора к нужной форме:

```
agent_data['Agent_name'] = agent_data['Agent'].apply(
    lambda x: x.split()[0][0] + ". " + x.split()[1] if isinstance(x, str) else np.NAN
)
```

Теперь надо сравнить имена операторов в этой и основной таблице. Опять воспользуемся группировкой:

```
agent_data.groupby(['Agent_name']).agg('first').head(20)
```

Agent_name	Agent	Supervisor	Manager	Tenure Bucket	Shift
A. Adams	Adriana Adams	Sophia Sato	John Smith	0-30	Morning
A. Aguilar	Andrew Aguilar	Nathan Patel	Emily Chen	On Job Training	Evening
A. Barnett	Adam Barnett	Abigail Suzuki	Jennifer Nguyen	On Job Training	Morning
A. Barrett	Andrew Barrett	Elijah Yamaguchi	John Smith	>90	Evening
A. Barron	Alexandra Barron	Austin Johnson	Jennifer Nguyen	On Job Training	Evening
A. Beck	Andrea Beck	Evelyn Kimura	Jennifer Nguyen	On Job Training	Evening
A. Bell	Amy Bell	Wyatt Kim	Jennifer Nguyen	On Job Training	Evening
A. Benjamin	Amanda Benjamin	Amelia Tanaka	Emily Chen	>90	Morning
A. Berry	Amy Berry	Elijah Yamaguchi	John Smith	>90	Evening
A. Booker	Angela Booker MD	Carter Park	Jennifer Nguyen	On Job Training	Evening
A. Booth	Anthony Booth	William Park	John Smith	31-60	Morning
A. Bowen	Anthony Bowen	Olivia Suzuki	Jennifer Nguyen	On Job Training	Morning
A. Bowers	Allen Bowers	Olivia Wang	John Smith	>90	Evening
A. Branch	Adrian Branch	Carter Park	Jennifer Nguyen	On Job Training	Morning
A. Brown	Amanda Brown	Emily Yamashita	John Smith	>90	Morning
A. Burke	Amy Burke	Sophia Chen	William Kim	On Job Training	Morning
A. Burton	Alicia Burton	Mia Patel	Emily Chen	0-30	Evening
A. Calderon	Amy Calderon	Elijah Yamaguchi	Michael Lee	31-60	Afternoon
A. Campbell	Alexis Campbell	Elijah Yamaguchi	Michael Lee	>90	Afternoon
A. Campos	Alisha Campos	Jacob Sato	Emily Chen	0-30	Evening

А вот такая же группировка по основной таблице:

```
data_123.groupby(['Agent_name']).agg('count').head(20)
```

Видно, что список полностью совпадает, то есть в двух таблицах информация по одним и тем же людям:

Agent_name	Id	Channel	category	Sub-category	Customer Remarks	Order_id	Issue_reported_Date	Issue_responded_Date	Su
A. Adams	215	215	215	215	70	175	215	215	21
A. Aguilar	31	31	31	31	11	19	31	31	31
A. Barnett	56	56	56	56	21	41	56	56	5
A. Barrett	50	50	50	50	16	35	50	50	5
A. Barron	25	25	25	25	9	18	25	25	2
A. Beck	70	70	70	70	17	51	70	70	7
A. Bell	20	20	20	20	10	18	20	20	2
A. Benjamin	64	64	64	64	22	48	64	64	6
A. Berry	135	135	135	135	62	89	135	135	1
A. Booker	21	21	21	21	3	19	21	21	21
A. Booth	177	177	177	177	60	133	177	177	1
A. Bowen	26	26	26	26	5	24	26	26	2
A. Bowers	126	126	126	126	35	101	126	126	1
A. Branch	43	43	43	43	19	40	43	43	4
A. Brown	197	197	197	197	55	155	197	197	1
A. Burke	80	80	80	80	28	69	80	80	8
A. Burton	29	29	29	29	9	24	29	29	2
A. Calderon	36	36	36	36	13	29	36	36	3
A. Campbell	117	117	117	117	39	91	117	117	11

A. Campos	95	95	95	95	31	75	95	95	95

Теперь можно приступать к соединению. Воспользуемся функцией join:

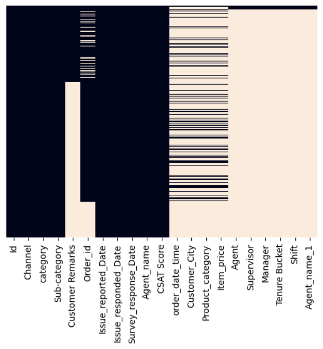
```
data_with_agents = data_with_orders.join(agent_data, rsuffix='_1', how='left')
data_with_agents.head()
```

index	Id	Channel	category	Sub-category	Customer Remarks	Order_id	Issue_reported_Date	Issue_responde
0	03f3aaf6-85c5-4eb9-af30-7ab7cc7b627d	Inbound	Cancellation	Not Needed	Food	b0a0dece-5fda-4c0f-a69c-e6bcde3de8d4	2023-08-29 08:02:00	2023-08-29 08:
1	f0442a16-b6d7-4fce-87fa-cbff2591176c	Outcall	Returns	Return request	V nicely	NaN	2023-03-08 18:38:00	2023-03-08 19:
2	99b7f6fa-a7b1-41a6-86ad-711ca8658fca	Inbound	Returns	Reverse Pickup Enquiry	Good	96e2ed8b-88b8-4768-a7e4-9468275c8085	2023-08-28 11:00:00	2023-08-28 00:
3	95dd50e0-bdbd-4e99-9278-e139a76a94d3	Inbound	Order Related	Delayed	Thank you for help	4340c8ed-d80f-4712-921a-4fea0d60dcc6	2023-05-08 23:42:00	2023-06-08 02:
4	2af35e04-d13b-43e9-9e69-358568f6174a	Inbound	Offers & Cashback	Affiliate Offers	Good	1549b443-c2c1-47b2-8608-cd4d5576d288	2023-01-08 11:13:00	2023-01-08 11:

При анализе первых строк датасета проблемы не видно, но давайте изобразим на графике пропущенные значения:

```
sns.heatmap(data_with_agents.isnull(), yticklabels=False, cbar=False)
```

Здесь мы видим явно аномальную картину:



Дело в том, что функция join по умолчанию проводит соединение по индексу двух датафреймов в качестве ключа. То есть в нашем случае, по номеру строки. Нам же нужно указать название определенного столбца. Для такого режима соединения больше подходит функция merge:

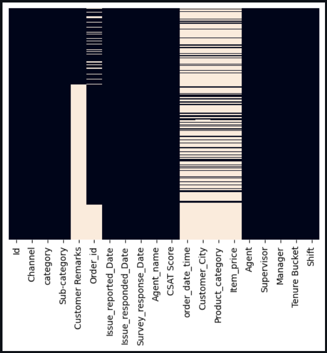
```
data_with_agents = data_with_orders.merge(agent_data, how='left', on='Agent_name', copy=False)
data_with_agents.head()
```

index	Id	Channel	category	Sub-category	Customer Remarks	Order_id	Issue_reported_Date	Issue_re
0	18f329f8-90af-4277-af78-acc1f41e8af2	Inbound	Returns	Reverse Pickup Enquiry	I AM satisfied for inconvenience	4cec934f-48c1-41da-9a54-7bea0248864d	2023-07-08 11:48:00	2023-07
1	21972b69-2e02-4d39-8643-d0c6153e6c04	Inbound	Returns	Missing	Good	NaN	2023-08-16 09:16:00	2023-08
2	97626f96-bd35-42ef-bf9e-07dfdde0400c	Inbound	Returns	Return request	Thank you sir for solving my return problem	c00bff95-5657-4acc-a8f2-2e242277b7d2	2023-09-08 19:19:00	2023-09
3	515adae8-3483-4e6a-b857-968dafdd04c5	Inbound	Order Related	Order status enquiry	Impossible to reach customer support.	21f4a92f-4a8e-4113-b9da-3af750f04f7c	2023-08-30 13:51:00	2023-08
4	f5a03787-5528-4bec-9fe1-9ed44bfcc2ec	Inbound	Feedback	UnProfessional Behaviour	from Shopzilla is amazing . THANKS A TON TO HIM FOR HELPING ME A PERSON WITH POLITE NATURE & BETTER ETIQUETTES LOTS OF HUGS & APPRECIATION TO HIM	9c8aa1c3-eb4d-4c65-8434-0af9fa8c75ff	2023-08-23 17:52:00	2023-08

Не доверяя глазам при анализе первых строк датасета, выведем диаграмму пропущенных значений:

```
sns.heatmap(data_with_agents.isnull(), yticklabels=False, cbar=False)
```

Картина сильно изменилась. Теперь отсутствующих значений совсем нет:



Однако, если отследить формы датасетов, можно заметить что в итоге получилось сильно больше строк, чем было в левой таблице. При проведении левого соединения такое может случиться только если в правой таблице есть несколько записей, соответствующих по ключу одной записи в левой таблице. Давайте проверим уникальность ключа в правой таблице:

```
agent_data.Agent_name.value_counts()
```

Получается, что после сокращения имен уникальность имени оператора потеряна:

```
J. Moore        6
A. Brown        4
M. Smith        4
D. Smith        4
K. Martin       4
               ..
J. Meadows      1
J. Edwards      1
J. Schroeder    1
J. Robertson    1
Z. Simpson      1
Name: Agent_name, Length: 1221, dtype: int64
```

Одному и тому же инициалу может соответствовать несколько разных полных имен:

```
agent_data[agent_data.Agent_name == 'J. Moore']
```

index	Agent	Supervisor	Manager	Tenure Bucket	Shift	Agent_name
468	Jacob Moore	Ava Wong	Jennifer Nguyen	On Job Training	Morning	J. Moore
529	Jenna Moore	Aiden Patel	John Smith	>90	Morning	J. Moore
542	Jennifer Moore	Nathan Patel	Jennifer Nguyen	On Job Training	Evening	J. Moore
612	Jon Moore	Ava Wong	William Kim	On Job Training	Evening	J. Moore
617	Jonathan Moore	Zoe Yamamoto	John Smith	>90	Morning	J. Moore
654	Juan Moore	Mason Gupta	Jennifer Nguyen	On Job Training	Morning	J. Moore

Что же делать в такой ситуации? Давайте посмотрим, какое количество записей в правой таблице соответствует каждой строке в левой. Для этого соединим левую таблицу со сверткой правой таблицы по именам:

```
data_with_agent_numbers = data_with_orders.merge(agent_data.Agent_name.value_counts(), how='left', left_on='Agent_name', right_
```

И выведем в этой таблице статистику по распределению количества строк:

```
data_with_agent_numbers.Agent_name_y.value_counts()
```

```
1    68749
2    11326
3     3695
4     1843
6      294
Name: Agent_name_y, dtype: int64
```

Мы видим, что все же абсолютному большинству строк основной таблицы соответствует единственная запись в таблице операторов. То есть дублирование строк - это скорее исключение. Поэтому мы можем добавить в датасет информацию об операторах в том случае, если сопоставление однозначное. А для остальных записей, которым соответствует несколько возможных имен, будут стоять пропуски. Как и положено в случае неопределенности данных.

Для этого удалим данные об имени оператора у тех строк, где соответствие множественное:

```
data_with_agent_numbers['Agent_name_x'] = data_with_agent_numbers.Agent_name_x.where(
    data_with_agent_numbers['Agent_name_y'] == 1.0, np.NAN
)
```

Теперь можно провести соединение:

```
data_with_agents = data_with_agent_numbers.merge(agent_data, how='left', left_on='Agent_name_x', right_on="Agent_name")
data_with_agents.info()
```

Видим, что теперь количество строк не увеличилось:

```
Int64Index: 85907 entries, 0 to 85906
Data columns (total 22 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   Id                   85907 non-null  object
 1   Channel              85907 non-null  object
 2   category             85907 non-null  object
 3   Sub-category         85907 non-null  object
 4   Customer Remarks     28425 non-null  object
 5   Order_id             66658 non-null  object
 6   Issue_reported_Date  85907 non-null  datetime64[ns]
 7   Issue_responded_Date 85907 non-null  datetime64[ns]
 8   Survey_response_Date 85907 non-null  datetime64[ns]
 9   Agent_name_x         68749 non-null  object
 10  CSAT Score           85907 non-null  int64
 11  order_date_time      16950 non-null  object
 12  Customer_City        16816 non-null  object
 13  Product_category     16932 non-null  object
 14  Item_price           16942 non-null  float64
 15  Agent_name_y         85907 non-null  int64
 16  Agent                68749 non-null  object
 17  Supervisor           68749 non-null  object
 18  Manager              68749 non-null  object
 19  Tenure Bucket        68749 non-null  object
 20  Shift                68749 non-null  object
 21  Agent_name           68749 non-null  object
dtypes: datetime64[ns](3), float64(1), int64(2), object(16)
memory usage: 15.1+ MB
```

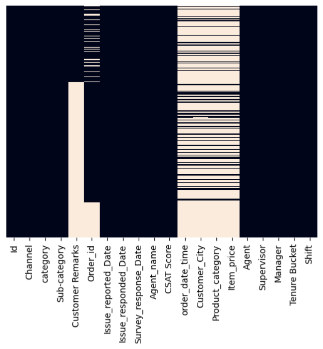
Осталось удалить лишние столбцы, которые мы вводили в таблицу для проведения служебных операций:

```
data_with_agents = data_with_agents.drop(['Agent_name_x', 'Agent_name_y', 'Agent_name'], axis=1)
```

И визуализируем пропуски в датасете:

```
sns.heatmap(data_with_agents.isnull(), yticklabels=False, cbar=False)
```

На данном графике видно, насколько мало данных мы "потеряли" из-за множественного совпадения имен:



В итоге мы получили датасет, содержащий максимум информации из исходных таблиц. Этот датасет пригоден к анализу и после очистки данных его можно использовать для машинного обучения.

Задания для самостоятельного выполнения

1. При выполнении вертикального объединения убедитесь в отсутствии дубликатов данных.
2. При горизонтальной интеграции первой и второй таблицы выведите по каждому столбцу гистограмму распределения или плотность распределения, чтобы убедиться, что признаки выражаются по соотносимым шкалам.
3. При выполнении третьего задания убедитесь более явно, что обильное количество пропущенных значений не является ошибкой объединения. Проведите больше, чем одну точечную проверку.
4. При выполнении третьего задания попробуйте использовать разные виды соединений. Как это отражается на структуре датасета?
5. Изучите документацию pandas в части описания методов merge и join.
6. Познакомьтесь с форматом представления дат strftime.

Контрольные вопросы

1. Какие ошибки могут произойти при горизонтальной интеграции данных?
2. Какими средствами можно проверить совпадение шкал измерения атрибута в разных датасетах?
3. Какие ошибки могут произойти при вертикальной интеграции данных?
4. Чем отличаются методы merge и join в библиотеке pandas?
5. Какие бывают виды соединения таблиц и в каких случая их нужно применять? Приведите по два примера на каждый вид объединения?

Дополнительные задания

1. В процессе выполнения лабораторной работы было сделано несколько решений, которые не определялись однозначно характером данных. То есть мы выбрали один способ, когда можно было сделать и по-другому. Идентифицируйте эти решения и проведите несколько альтернативных вариантов интеграции датасета из данной работы. Сравните полученные результаты.
2. Найдите несколько (3-4) датасетов в открытых источниках по оной и той же проблеме. Важно, чтобы датасеты были из разных источников. Попытайтесь провести вертикальную интеграцию этих датасетов.